Real Estate on Purpose — Hub Architecture

Audience: business partners and technical reviewers who need to understand how the REOP CRM ("the Hub") actually works under the hood — not a marketing summary, but a faithful map of what runs, when, and why.

Last updated: May 14, 2026, against `main` branch commit `02097fb` (after the Pam pipeline-stage migration).

Table of contents

---

1. What the Hub is for

The REOP Hub is a real-estate-agent productivity platform that wraps three jobs into one place:

1. **Stay in touch with the sphere** — make sure every person an agent knows hears from them on a predictable cadence (the "SphereSync" rotation). This is the prevention-of-leaks layer.
2. **Move active opportunities forward** — track conversations that became real deals, log activity, and don't let any of them go stale (the Pipeline + AI Coach).
3. **Hold the agent accountable to their own numbers** — weekly check-ins, year-to-date goals, and a coach (human and AI) that reflects back where they are vs where they said they wanted to be (the Scoreboard).

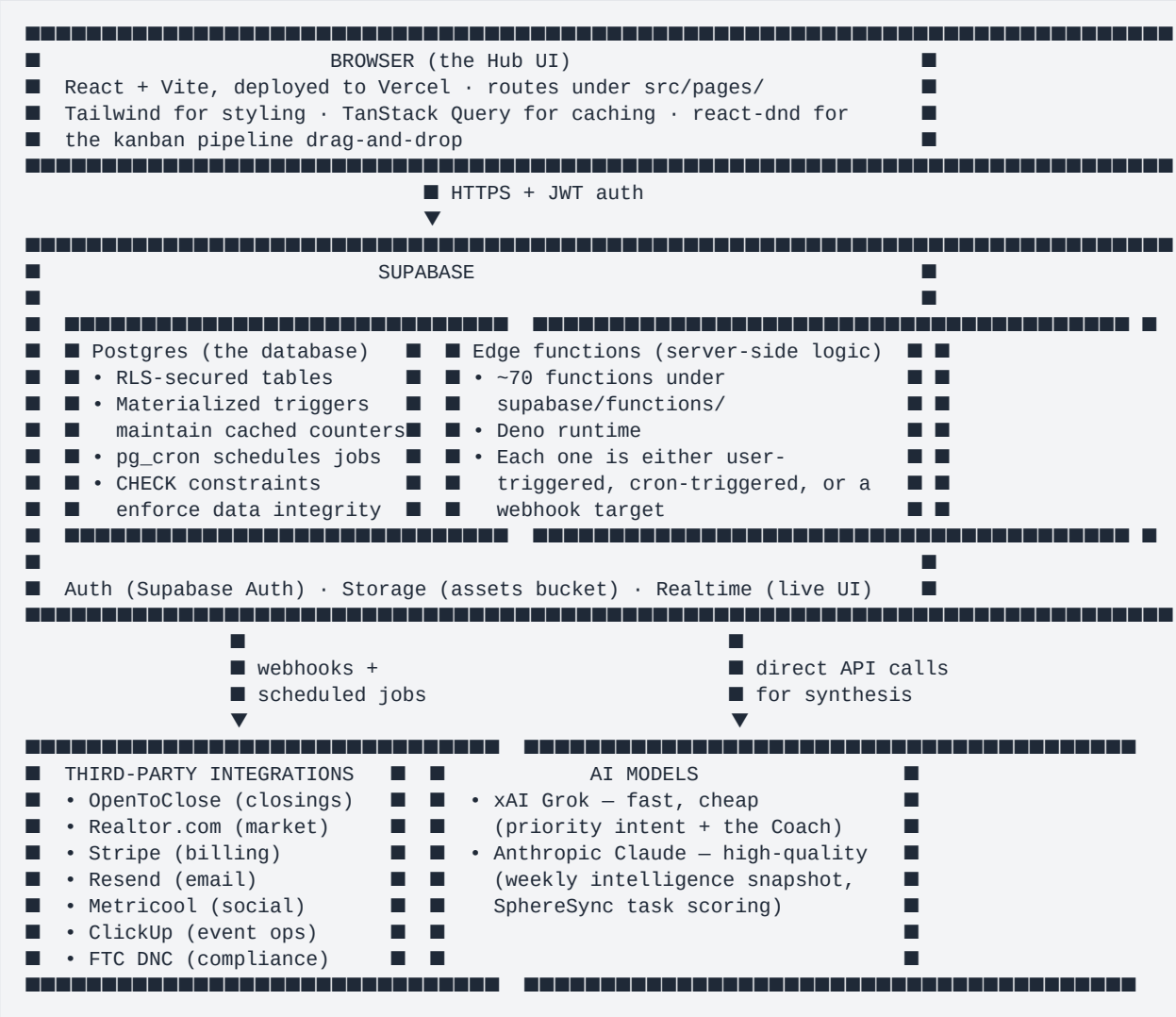
Everything else in the Hub — the newsletter, delight gifts, events, the support knowledge base — exists to feed or support one of those three loops.

The product's design philosophy is **agents shouldn't have to think about what to do next**. The Hub computes that for them: the SphereSync rotation says "this week, call last-names starting with M and S, text last-names starting with X"; the priority score says "these 8 contacts are the warmest right now"; the AI Coach says "the most important thing in your next hour is to call Brian — he texted you today and you've had 3 real conversations with him this month."

The agent's job is to act on what the Hub tells them, log what happened, and let the cycle repeat.

---

2. System architecture at 10,000 feet



Two practical things to know:

- **The database is the source of truth.** Edge functions, AI models, and integrations all read from and write to Postgres. The browser does too. There is no separate "service layer."
- **The AI has a budget.** Grok handles the high-volume, low-stakes work (which contact is highest intent right now, what's the agent's next-hour move). Claude handles the once-a-week, high-stakes synthesis (sphere health score, weekly priorities narrative). This keeps the monthly bill bounded.

---

3. The data model

Roughly 60 tables, but five do most of the work. Everything else feeds these:

Table	What it represents	Approx columns
`contacts`	Every person in an agent's sphere	~90, including identity, address, category (last-name initial), `priority_score`, `priority_components` (jsonb), `priority_signals` (jsonb), birthdays/anniversaries (for Delight), DNC status
`opportunities`	Active or closed real-estate deals tied to a contact	~40, including `stage` (Pam's 7 + lost + null), `opportunity_type` (buyer/seller/referral), `next_step_title`, `next_step_due_date`, financial fields, AI-scored deal probability
`spheresync_tasks`	Weekly call/text tasks generated by the cadence rotation	~15, including `task_type` ('call'/'text'), `week_number`, `year`, `completed`, AI fields like `ai_priority_score`, `ai_reason`, `ai_talking_points`
`contact_activities`	Every logged touch — calls, texts, emails, gifts, meetings, notes	~10, including `activity_type`, `outcome` (e.g. 'conversation' vs no-outcome dial attempt), `notes`, timestamps
`coaching_submissions`	Weekly agent self-reported numbers (conversations, dials, closings, etc.)	~30, including the v2 ratings (energy/focus/confidence 1–5), focus areas, weekly metrics

Three other tables are critical for AI:

- `agent_coaching_state` (one row per agent, rewritten frequently) — the real-time "Coach state" cache, written by the `ai-coach-agent` edge function on a tick cadence.
- `agent_intelligence_snapshots` (one row per agent per ISO week) — the slow, dashboard-grade weekly synthesis written by Claude.
- `opportunity_stage_history` — every stage transition logged with timestamps. The audit trail.

Two supporting tables for engagement:
- `events` + `event_rsvps` — for in-person events the agent hosts.
- `newsletter_campaigns` + `newsletter_schedules` — for the recurring email newsletter.

The full schema is reflected in `src/integrations/supabase/types.ts` (auto-generated from Postgres).

Row-Level Security (RLS)

Every table that holds per-agent data has an RLS policy: **agents see only their own rows; admins see everything**. The policy is enforced by Postgres on every query — even a bug in the React code that tried to fetch another agent's

contacts would return zero rows. This is the foundational security guarantee.

The `get_current_user_role()` Postgres function reads the role from `auth.users.raw_app_meta_data` and is used by every RLS policy to grant admin access.

---

4. The Hub, page by page

The Hub's main sidebar has 12 routes. Each is gated by tier (`core` / `managed` / `agent` / `admin`) via `src/hooks/useFeatureAccess.ts`. Below is what each one does, what it reads from, and what an agent does there.

Screenshot: The Dashboard `/` rendered on localhost — sidebar with all sections, hero greeting card, four KPI cells, three module tiles (Sphere touches / Pipeline value / Delight sent), Recent activity + Upcoming events split, and the Jump back in shortcut grid.

Route	Tier	What it does	Reads from
`/` (Dashboard)	core	Mission-control on login. Personalized hero greeting, four YTD KPIs (sphere touches, pipeline value, closings, conversations YTD), weekly streak card, three module tiles, recent activity feed, upcoming events, shortcuts	`useUserProfile`, `useSphereSyncTasks`, `usePipeline`, `useDelightOpportunities`, direct queries to `opportunities` and `coaching_submissions`
`/spheresync-tasks`	core	The "what to do now" surface — three tabs (Priorities / Cadence / History). Lists who to reach this week with reasons	`useSphereSyncTasks`, `usePrioritizedQueue`, opens `ContactQuickSheet`
`/pipeline`	core	Kanban board with Pam's 7 stages (Conversation Active → Closed) plus a sphere-only state. Drag cards forward, AI scores probabilities	`usePipeline`
`/scoreboard`	core	Weekly check-in form + KPIs + 6-month trajectory chart + growth goals. Auto-opens check-in modal on `?nudge=…`	`useCoachingSubmissions`, `usePersonalMetrics`, `useWeeklyStreak`, `useCoachingGoals`

Route	Tier	What it does	Reads from
`/database`	core	Contact list with temperature pills, segmentation, search, CSV import/export, DNC handling	`useContacts`, `usePrioritizedContacts`, `useDatabaseStats`
`/events`	core	Event planning + RSVP tracking. Public link per event at `/event/:slug` (unauthenticated)	`useEvents`
`/newsletter`	core	Email-newsletter command center. AI-generation, templates, recurring schedules, analytics	`useNewsletterAnalytics`, `useNewsletterTemplates`
`/delight`	open	Birthdays, anniversaries, gift suggestions, gifting history	`useDelightOpportunities`, `useGiftHistory`
`/support`	core	Knowledge base + ticket system. Tier-gated articles	`useSupportArticles`, `useArticleSearch`
`/settings`	open	Configuration center — profile, brand, goals, notifications, billing, security, data export	`useUserProfile`, `useAgentMarketingSettings`, `useCoachingGoals`, `useNotificationPrefs`
`/transactions`	**admin**	Closed/active transactions with OpenToClose sync. Surfaces deal financials, admin-only	`useTransactions`
`/social-scheduler`	agent	Five-tab social media hub (Compose/Calendar/Connect/Analytics/Overview). All flows through Metricool	`useSocialAccounts`, `useSocialPosts`, `useSocialAnalytics`
`/resources`	core	Downloadable file library (PDFs, scripts, training)	`useResources`

Public (no auth required): `/event/:slug` (RSVP page), `/support/articles/:slug` (KB articles), `/auth`, `/pricing`.

Screenshot: The Settings page `/settings` rendered on localhost — sticky-rail nav with Profile/Brand/Goals/Notifications/Billing/Security/Data sections, profile form with first/last name, email, phone, brokerage, license fields.

5. Cadence — how SphereSync schedules outreach

SphereSync solves a specific problem: **how do you make sure every contact in an agent's sphere gets a touch on a predictable rhythm, without an agent having to remember who they haven't called in a while?**

The letter rotation

Source of truth: `src/utils/sphereSyncLogic.ts`.

Two lookup tables drive everything:

- `SPHERESYNC_CALLS` — maps every ISO week (1–52) to a **pair of last-name initials** to call this week. Pairs are deliberately constructed to mix one high-frequency English surname letter (S, M, B, C, T, W) with one low-frequency letter (Q, X, Y, Z). Example: week 1 = `['S', 'Q']`, week 2 = `['M', 'X']`, week 3 = `['B', 'Y']`, and so on.
- `SPHERESYNC_TEXTS` — maps every ISO week to **one letter** to text. Texting is lower-friction so it rotates through all 26 letters over the year, one per week.

Why this design (and not date-based "call this contact again in 90 days")? The header comment in `sphereSyncLogic.ts` is explicit: letter rotation gives **predictable, even weekly load** that is a pure function of (ISO week × surname). A date-based "next-touch-due" cadence collapses under bursty imports — 1,000 contacts loaded on a Monday all become overdue together on the same day 90 days later. Letter rotation is stateless and self-balancing.

A contact is assigned to this week's call or text list based on their `category` column, which is auto-derived from `UPPER(last_name[0])`. So `Smith` → category `S` → called whenever the week's call pair includes `S`. Across a year, every contact gets ~2 calls and ~1 text.

Weekly task generation

Every Monday at 05:30 UTC, a pg_cron job named `spheresync-weekly-task-generation` fires the `spheresync-generate-tasks` edge function for every agent in the system:

1. The function loads the agent's contacts, filtered to those matching this week's call letters and text letter.
2. It inserts rows into `spheresync_tasks` for each one — typically dozens of call tasks and a handful of text tasks per agent per week.
3. It calls **Claude** (`claude-3-5-sonnet-20241022`) in batches to score each task 1–10 with a reason and two suggested talking points, then writes those into `ai_priority_score`, `ai_reason`, and `ai_talking_points` columns.
4. It logs the entire run to `spheresync_run_logs`.
5. Fifteen minutes later (Monday 05:45 UTC), a separate cron `spheresync-monday-emails` sends each agent an email listing their tasks. Tuesday at 06:30 UTC, `spheresync-tuesday-backup-emails` retries any emails that failed.

If the cron is missed entirely (or an admin imports new contacts mid-week), `useSphereSyncTasks` will detect missing tasks and trigger generation on demand.

The priority queue — three bands

When an agent opens `/spheresync-tasks` and looks at the **Priorities** tab, what they see is **not** the AI Coach's ranked list. It's a deterministic 3-band hierarchy computed client-side by `usePrioritizedQueue`:

Band	Who lands here	Cap	Sort order
1 — Pipeline	Anyone with an active opportunity (no `actual_close_date`, outcome not won/lost/withdrawn)	8	Most overdue first (sorted ascending by `last_activity_date`)
2 — Cadence	Contacts whose last-name initial matches this week's rotation, *not already in Band 1*	12	Same — most overdue first
3 — Engagement	Contacts who engaged with marketing in the last 30 days (sent a gift, RSVP'd to an event), *not already in Bands 1 or 2*	5	Most recent engagement first

A higher band always outranks a lower band regardless of score. The agent never sees the band labels — they see a one-line `reason` chip that summarizes why the contact is in front of them. Examples:
- *"In your pipeline · Active opportunity · last touch 5d ago"*
- *"This week's rotation (M) · 14d ago"*
- *"Sent a gift 8d ago"*

This is the design's philosophy in action: the math is hidden, the call to action is visible.

What happens when an agent marks a task done

A single click writes to multiple tables in sequence (`useSphereSyncTasks.updateTask`):

1. `spheresync_tasks.completed = true` (and `completed_at = now()`).
2. A new row in `contact_activities` with `activity_type = 'call'` or `'text'`.
3. A Postgres trigger (`trg_refresh_contact_channel_last_touch`) updates the contact's `last_activity_date` and channel-specific last-touch columns.
4. A fire-and-forget call to the `compute-priority-scores` edge function for that contact, so the Coach view reflects the fresh touch within a minute or two.
5. If the contact has an active opportunity, a parallel row is also written into `opportunity_activities` so the pipeline timeline captures it.

Handoff from SphereSync to Pipeline

Manual. The Hub does not auto-promote a SphereSync contact into the pipeline. The agent decides "this person is now a real opportunity" by opening the Add Opportunity dialog and filling it out. Once an opportunity row exists

in the right state, the contact promotes itself to Band 1 of the priority queue automatically (and stops appearing in Band 2's cadence list, via the dedupe rule).

---

6. Temperature — the contact priority score

Every contact has a single **0–100 priority score** (`contacts.priority_score`) that drives every "who's hot right now?" surface in the app. Here's how it's computed.

The formula

The score is a weighted blend of four components. Weights depend on whether the contact has an active opportunity:

Component	Weight (with opp)	Weight (without opp)
`relationship`	35%	**50%**
`pipeline`	30%	0% *(redistributed)*
`intent`	25%	**40%**
`flags`	10%	10%

When there's no opportunity, the pipeline weight redistributes to relationship and intent — because for a sphere-only contact, the relationship freshness and AI-assessed intent are what matter.

What each component measures

- `relationship` — pure freshness curve based on `last_activity_date`. 92 if touched in the last 30 days, 78 if 30–60d, 58 if 60–90d, 38 if 90–180d, 15 beyond. Plus small bonuses for activity density (+8 if 3+ activities in the last 30 days). Never-touched gets a 30 baseline.
- `pipeline` — only non-null if the contact has an active opportunity. Uses `opportunities.ai_deal_probability × 100` if AI-scored, else a stage-based baseline (Conversation active = 20, Opportunity identified = 35, Consultation completed = 55, … Under contract = 88, Closed = 100). Penalties for stagnation: –18 if >30 days in current stage, –8 if >14 days.
- `intent` — **AI-synthesized 0–100 score from Grok** (`grok-4-1-fast-reasoning`). The model gets a batch of 20 contacts at a time, sees their category, ZIP, life events, engagement counts, market pulse, and active opportunity summary, and returns a buying-intent score with a one-sentence reasoning. On API failure, falls back to neutral 40.
- `flags` — baseline 30, plus +25 for each of: agent-set "watch this contact" flag, buyer pre-approval status = approved, contact tag containing "VIP". +10 if motivation score ≥ 7. Capped at 100.

Recompute cadence — tiered crons

The system doesn't recompute every contact every day — that would be wasteful and expensive. Instead, contacts are rescored on a tiered schedule by tier:

Tier	Score range	Cron	Rationale
Hot (and unscored)	≥ 60	Daily 05:00 UTC	Active leads, recompute often
Warm	40–59	Sundays 06:00 UTC	Weekly is enough
Cool	20–39	Wednesdays 07:00 UTC	Biweekly
Cold	< 20	1st of month 08:00 UTC	Monthly is plenty

Each tier's cron caps the batch at 200 contacts per tick. Additionally, two AFTER triggers (on `contact_activities` INSERT and `opportunities` UPDATE-of-stage) fire a single-contact rescore as fire-and-forget, so logging activity reflects almost immediately.

Tiers (the "temperature" labels)

The numeric score maps to a visible tier in `usePrioritizedContacts.ts`:

Tier	Score	Used for
Urgent	80–100	The very top of the Database filter
Hot	60–79	Dashboard "Hot leads" counter, Database segmentation
Warm	40–59	Middle band
Cool	20–39	Long-tail follow-up
Cold	0–19	Annual touch only
Unscored	null	Recently imported, awaiting first cron pass

Each tier has a color (red/amber/yellow/cool/gray) used consistently across the Database page filter, the priority queue, and the Coach decisions.

`priority_reasoning` and `priority_signals`

Every score is accompanied by:

- `priority_reasoning` — a one-sentence English explanation, written by Grok (or a deterministic fallback if the AI call fails). Example: *"Active deal in offer submitted, momentum strong."* / *"Overdue touch — last activity 47 days ago."*
- `priority_signals` (jsonb) — the audit trail of raw inputs: days since last activity, activity counts (30d / 90d), life event, market ZIP, AI key signals (2–3 short phrases). Available so the agent can see *why* the AI rated the contact the way it did.

The "watch" flag

`contacts.priority_watch_flag` is the agent's manual override — "this person is important to me, regardless of what the score says." Set via a star icon in the contact drawer. Contributes +25 to the `flags` component, so a watched contact will reliably score higher than an unwatched peer.

Where the score shows up

- **Database page** — the Temperature filter rail (5 tiers + unscored) and the per-row chip.
- **SphereSync ContactQuickSheet** — drawer shows the score, watch-flag star, the AI reasoning, and a stacked bar visualizing the 4 component contributions.
- **Dashboard** — "Hot Leads" counter (count of score ≥ 60).
- **AI Coach** — filters `priority_score ≥ 70` to surface "hot list" suggestions; sorts contact recommendations by score descending.

---

7. The Pipeline — opportunities and stages

The Pipeline (`/pipeline`) is where an agent tracks real estate transactions in flight. Each row in the `opportunities` table is one tracked deal tied to one contact.

The 7 stages

Adopted from Pam's April 10, 2026 technical brief, applied as a database migration on May 14, 2026. The 7 stages are **universal** — they apply equally to buyer and seller deals (the `opportunity_type` field is preserved as an independent badge but no longer drives different stage tracks):

#	Stage	Meaning
1	**Conversation active**	First chat — just entered the pipeline
2	**Opportunity identified**	Confirmed they're a real buyer/seller
3	**Consultation completed**	Buyer consult or listing presentation done
4	**Client secured**	Buyer-rep or listing agreement signed
5	**Active opportunity**	Buyer touring or listing live on the market
6	**Under contract**	Offer accepted, in escrow
7	**Closed**	Transaction complete

Plus:
- **Lost** — terminal state, off the kanban board by default (filter-accessible).
- **NULL stage** — sphere-only opportunity. The row exists in the table (preserving notes, dates, value) but doesn't appear on the kanban. Used when an opportunity becomes inactive but the relationship continues.

The 7-stage list is the single source of truth in `src/config/pipelineStages.ts`. A Postgres CHECK constraint on `opportunities.stage` enforces it server-side, so no rogue value can be written.

Kanban behavior

The board (`PipelineBoard` component) renders 7 columns side-by-side, each ~200px wide. On screens narrower than ~1400px the row scrolls horizontally. Each column shows:

- An ALL-CAPS short label (e.g. "CONVERSATION ACTIVE", "OPPORTUNITY", "CONSULT DONE", "CLIENT SECURED", "ACTIVE", "UNDER CONTRACT", "CLOSED").
- A color dot + accent line in the column's hue (cool → warm → green progression).
- The count of opportunities in that stage and the total dollar volume.
- Stacked cards showing each opportunity: contact name, type badge (Buyer/Seller/Referral), DNC warning if applicable, next-step text, due date (red if overdue), dollar value.

Agents move cards two ways:
1. **Drag and drop** on desktop (HTML5 backend) or mobile (touch backend via `react-dnd-multi-backend`).
2. **A "■" menu** on each card with the 7 stages listed — for accessibility and small viewports where drag is fiddly.

Either method writes the new stage value to `opportunities.stage`, which fires the `trg_log_opportunity_stage_change` trigger to record the transition in `opportunity_stage_history`. The Coach is also flagged "dirty" so the next 2-hour tick picks up the change.

The opportunity detail drawer

Clicking a card opens a drawer (`OpportunityDetailV2`) with:

- **Snapshot** — type, stage, AI-scored deal probability, days in current stage, value, expected close date.
- **Next step block** — a forced "what's the next thing you'll do?" field with a due date. The product's philosophy is that **every opportunity must always have a next step** — if one isn't set, the Coach flags it.
- **Quick actions** — Call (with DNC respect), Text, Email, Log activity, Send listing.
- **Activity timeline** — every logged interaction in chronological order.
- **AI insights** — when present: deal probability with a confidence range, risk flags (e.g. "30 days in same stage"), suggested next action.
- **Coach blurb** — when the open contact matches the Coach's `next_hour` recommendation, a purple call-out shows the Coach's reasoning and a suggested opener.

Stale and overdue detection

Three rules fire silently and surface as visual cues:

- **Stale** — `last_activity_date` > 30 days. Card shows an amber "stale" pill.
- **Overdue next step** — `next_step_due_date` < today. Card's due date renders in red.
- **No next step** — `next_step_title` is null. The Coach raises an `opportunity_no_next_step` alert.

A daily cron also recomputes a derived `is_stale` boolean and `days_in_current_stage` integer.

8. The AI Coach

The AI Coach is the part of the Hub that says "this is what to do right now" — a single recommendation per agent, refreshed throughout the day.

Two intelligence layers

The Coach is actually two systems with different cost/freshness tradeoffs:

Layer	Table	Model	Cadence	Cost
Real-time	`agent_coaching_state` (one row per agent, rewritten on each tick)	xAI Grok (`grok-4-1-fast-reasoning`)	Nightly full refresh + every 2 hours during workdays for "dirty" agents	Cheap
Weekly synthesis	`agent_intelligence_snapshots` (one row per agent per ISO week)	Anthropic Claude (`claude-3-5-sonnet-20241022`)	Once per week	Expensive but rare

The split exists because the two have opposite jobs. The real-time Coach must react within hours of an agent logging a call — that's Grok territory. The weekly synthesis is a Monday-morning "here's your week ahead" narrative the agent reads once — worth spending Claude tokens on.

What the real-time Coach writes

When `ai-coach-agent` runs, it produces five things and writes them into `agent_coaching_state`:

- `next_hour` — a single recommendation: contact name, action (call/text/email/meet/...), urgency level, plain-English reasoning, a suggested opening sentence to send, and up to 3 context chips (e.g. "3 real convos 30d", "text today", "last call 10 days ago").
- `today_list` — a ranked array of 5–8 contacts to tackle today, each with its own action and reasoning.
- `week_narrative` — five short sentences: GCI pace, pipeline story, sphere story, top risk, top win.
- `alerts` — up to 8 alerts of types:
- `overdue_touch` — >60 days since a real conversation
- `stuck_deal` — >21 days in same stage (urgent if >$500k)
- `life_event` — life event captured + no contact in 14 days
- `high_priority_ignored` — score ≥70 and >14 days untouched
- `opportunity_no_next_step` — opportunity has no `next_step_title`
- `chat_context` — a context bundle used by the in-app chat assistant (when wired up).

Plus metadata: `generated_at`, `model`, `tokens_in`, `tokens_out`, `run_ms`, `dirty` flag.

How the Coach gets triggered

Three paths:

1. `coach-nightly-full` — daily 05:00 UTC pg_cron. Refreshes every agent unconditionally with `write_tasks: true` (i.e. allowed to create new Coach-authored SphereSync or Pipeline tasks).
2. `coach-workday-quick` — every 2 hours from 13:00 to 23:00 UTC. Only refreshes "dirty" agents (those whose state needs updating). Skips task writing.
3. **On-demand** — the "Refresh" button in the `CoachTrustBar` calls the edge function directly.

The "dirty" flag is set automatically by two database triggers: any time a `contact_activities` row is inserted OR an opportunity's stage changes, the corresponding agent's `agent_coaching_state.dirty` becomes true. The next quick-refresh tick picks them up.

What the weekly synthesis produces

`generate-agent-intelligence` runs once a week per agent and writes:

- `sphere_health` — a 0–100 score deterministically computed (`100 - (overdue90 / total × 50) - (overdue30 / total × 30)`) plus a Claude-written summary, plus key stats (total contacts, contacts never touched, overdue 30d/90d).
- `top_opportunities` — Claude's pick of the 3–5 most important pipeline rows this week with `next_action` per each.
- `market_pulse` — neighborhood-level summary from Realtor.com data.
- `weekly_priorities` — a ranked list of contacts to focus on, with talking points.
- `coaching_context` — week-trend (improving/declining/steady) plus a one-sentence observation, plus 4-week averages of dials and conversations.

Honest caveat about wiring

A few Coach outputs are computed and stored but **not yet rendered in the live UI**:

- `today_list`, `week_narrative`, and the global `alerts[]` array are all populated correctly but no page currently consumes them.
- `AgentIntelligenceWidget` (which would render `sphere_health` and `weekly_priorities` on the dashboard) is built but not mounted.
- `useCoachTasks` / `useDismissCoachTask` hooks for Coach-authored task management are built but not yet imported by any component.

The Coach is **running** (the cron is firing, the data is being written), but only two surfaces consume its output today:

- `CoachContactBlurb` — when an agent opens an opportunity drawer, if the open contact matches the Coach's current `next_hour` recommendation, a purple call-out shows the Coach's reasoning and the suggested opener.
- `CoachTrustBar` in the SphereSync Priorities tab — shows "Coach updated N minutes ago" with a refresh button. Doesn't surface the recommendations themselves.

For the Priorities tab itself, the visible ranked list comes from the **deterministic** `usePrioritizedQueue` (the 3-band system documented above), not from the Coach. The Coach's `today_list` is computed but parked — a design decision made in the May rewrite to favor predictability over AI variability for the daily call list.

This is fixable: wiring `today_list` into a dashboard widget and `alerts[]` into a notifications strip is a near-term TODO. Worth flagging to partners as "ready to surface but not yet surfaced."

---

9. Coaching & the Success Scoreboard

The Scoreboard (`/scoreboard`) is the agent's accountability surface. It's where they log weekly numbers and see whether they're tracking toward their annual goals.

The weekly check-in

Every week, the agent submits a check-in via a modal that captures:

The numbers (in `coaching_submissions`):
- Conversations (real two-way talks)
- Dials made (call attempts, including unanswered)
- Appointments set / appointments held
- Agreements signed (buyer rep or listing)
- Offers made/accepted
- Closings + closing amount
- Database size

The qualitative (also in `coaching_submissions`):
- Wins (free-text)
- Challenges (free-text)
- Must-do task for next week (free-text)

The self-assessment (v2 fields added May 2026):
- Energy rating (1–5)
- Focus rating (1–5)
- Confidence rating (1–5)
- Focus areas (multi-select: calling, sphere, pipeline, event prep, delight, listings)

The streak

`useWeeklyStreak` counts consecutive weeks an agent has submitted a check-in. Rendered as:

- A small badge in the Scoreboard hero ("6 weeks").
- A 7-square strip on the Dashboard's StreakCard, where hit weeks are teal, missed are gray, and the current week is dark blue with a green pulse dot. The current `feat/pipeline-stages-pam` branch redefined "hit" to mean *every SphereSync call task assigned that week was completed* (not just "submitted a check-in"), giving the streak a sharper meaning.

Annual goals

Set in `/settings` → Annual goals:

- Annual GCI ($)

- Annual closings (count)
- Annual conversations (count)

These drive the Dashboard's hero KPI trends (`X% of annual`) and the Scoreboard's progress bars. When left blank, the relevant UI shows "Set goal →" instead of fabricating a target.

Reminders

Two paths nudge agents to actually do their check-in:

- **Email reminder** — `coaching-reminder` cron fires Wednesday 18:00 UTC (overriding an older Thursday slot per the latest migration). Sends an email to any agent who hasn't submitted for the current ISO week.
- **In-app nudge** — `coaching-weekly-nudge` cron runs Wed/Thu/Fri and creates `agent_action_items` rows visible on the Dashboard alert strip. Escalates: Wed = `low` priority, Thu = `medium`, Fri = `high`. Respects each agent's notification preferences (`notify_in_app`, `quiet_hours_start/end`, `timezone`) — set in `/settings` → Notifications.

---

10. Newsletter, Delight, and Events

These three feed into the priority and Coach systems by generating engagement signals.

Newsletter

Two modes:

1. **One-off campaigns** — agent composes (or AI-generates), schedules, sends. Backed by `newsletter_campaigns` table; dispatched by a `newsletter-scheduled-send` cron running every 5 minutes (picks up campaigns whose `scheduled_at` has passed).
2. **Recurring schedules** — agent sets a "monthly newsletter on the 15th" or "weekly on Tuesdays" via `NewsletterScheduleManager`. Backed by `newsletter_schedules` table; dispatched by `newsletter-recurring-dispatch-hourly` cron at :05 of every hour (picks up schedules where `next_send_at <= NOW()`).

The Hub also has a "Newsletter cadence banner" on the Dashboard that surfaces when an agent's rhythm slips (e.g. last send > 35 days ago, no recurring schedule active). It hides itself when cadence is on track.

AI generation (`generate-ai-newsletter`) uses the agent's brand settings — primary color, headshot, signature, scheduling URL — to produce a personalized newsletter draft. The agent reviews and edits before sending.

Delight

The Delight system (`/delight`) tracks three occasion types per contact: birthday, spouse birthday, home anniversary. The data is captured during onboarding or back-filled from public records.

`useDelightOpportunities(windowDays)` computes upcoming occasions within the next N days. Rendered as cards on the Delight page and as a row on the Dashboard ("Birthdays this week"). Each card has a "Send gift" action that opens a flow to AI-suggest a gift (via `delight-suggest-gift`, which knows the contact's category, location, and gift history).

Logged gifts write to `contact_activities` with `activity_type = 'gift'`. This:
- Counts toward the Engagement band of the priority queue (Band 3 — sent a gift in the last 30 days).
- Is reflected in the Dashboard's "Delight sent · MTD" module.
- Marks the contact for priority rescore (gifts move the relationship score up).

A daily cron `delight-daily-nudge` looks ahead 7/1/0 days and creates `agent_action_items` for agents whose contacts have birthdays/anniversaries coming up.

Events

Events (`/events`) are agent-hosted in-person gatherings — open houses, client appreciation parties, market briefings, charity drives.

- Each event has a public RSVP page at `/event/:slug` (no auth required) that auto-brands with the agent's colors, headshot, and logo.
- RSVPs write to `event_rsvps` and a database trigger (`trg_sync_rsvp_to_contacts`) auto-creates or links the RSVP submitter to the agent's `contacts` table.
- The same trigger maintains a cached `events.rsvp_count` so the agent doesn't need to recompute it.
- A separate `event-email-scheduler` edge function sends RSVP confirmations, day-before reminders, and post-event thank-yous.

Like gifts, RSVPs count toward the Engagement band of the priority queue.

---

11. Scheduled jobs (what runs automatically)

The Hub has **17 active scheduled jobs** plus dozens of database triggers. Here's the canonical list — UTC times, grouped by purpose.

Sphere maintenance

Job	Schedule	Plain English
`spheresync-weekly-task-generation`	Mon 05:30	Generate weekly call/text tasks for every agent
`spheresync-monday-emails`	Mon 05:45	Email each agent their task list
`spheresync-tuesday-backup-emails`	Tue 06:30	Retry any failed Monday emails
`dnc-monthly-check`	1st of month 05:00	Re-verify every contact phone number against the FTC DNC registry

Priority rescoring (tiered)

Job	Schedule	What it rescores
`priority-rescore-hot-daily`	Daily 05:00	Unscored + Hot tier (≥60) contacts
`priority-rescore-warm-weekly`	Sun 06:00	Warm tier (40–59)
`priority-rescore-cool-biweekly`	Wed 07:00	Cool tier (20–39)
`priority-rescore-cold-monthly`	1st of month 08:00	Cold tier (<20)

AI Coach

Job	Schedule	What it does
`coach-nightly-full`	Daily 05:00	Full Grok refresh for every agent; allowed to create Coach-authored tasks
`coach-workday-quick`	Every 2h, 13:00–23:00	Refresh "dirty" agents only; no task writes
`coach-task-ttl-sweep`	Daily 04:00	Auto-dismiss Coach tasks older than 30 days

Coaching reminders & nudges

Job	Schedule	What it does
`coaching-thursday-reminder` (alias `coaching-reminder`)	Wed 18:00	Emails agents who haven't submitted their weekly check-in
`coaching-weekly-nudge`	Wed/Thu/Fri 13:00	Creates escalating in-app `agent_action_items` for missing check-ins (respects quiet hours + opt-out)

Newsletter dispatch

Job	Schedule	What it does
`newsletter-scheduled-send`	Every 5 min	Send one-off campaigns whose time has come
`newsletter-recurring-dispatch-hourly`	Hourly at :05	Dispatch recurring schedules (weekly/monthly)

External integrations

Job	Schedule	What it does
`opentoclose-daily-sync`	Daily 06:00	Pull closed/active transactions from Open To Close
`sync-realtor-market-data-monthly`	18th of month 06:00	Pull ZIP-level market CSV from Realtor.com into `market_stats`
`clickup-sync-event-tasks-every-2h`	Every 2h	Reconcile ClickUp event-prep tasks

Material database triggers

Beyond cron, the database itself has triggers that maintain cached state:

- `trg_log_opportunity_stage_change` — every stage change is logged to `opportunity_stage_history`.
- `trg_priority_rescore_after_activity` / `_after_stage_change` — marks a contact for the next priority cron pass.
- `trg_coaching_dirty_after_activity` / `_after_stage_change` — flags an agent's Coach state as needing refresh.
- `trg_refresh_contact_channel_last_touch` — keeps `contacts.last_activity_date` and channel-specific last-touch columns in sync with `contact_activities` writes.
- `trg_refresh_contact_pipeline_active` — keeps a cached `contacts.has_active_opportunity` flag in sync.
- `trg_sync_rsvp_to_contacts` — auto-creates contacts when someone RSVPs to an event.
- `set_contact_category` — auto-derives `category = UPPER(last_name[0])` on insert.
- `normalize_contact_phone_trigger` — normalizes phone numbers to E.164 on insert/update.
- A family of audit triggers writes to `security_audit_logs` for compliance.
- `on_auth_user_created` — creates a `profiles` row + default settings whenever a new user signs up.

Heaviest moments in the schedule

Two times of day are the system's "Monday morning" equivalents:

- **Monday 05:30–05:45 UTC** — the SphereSync weekly cycle: tasks generate, emails go out. Agents land in their inbox/dashboard with a fresh week's task list.
- **Daily 05:00 UTC** — the Coach's full nightly refresh + Hot-tier rescores. By the time agents are awake on the US East Coast, the Coach's state for that morning is fresh.

Together these are the foundation of the daily "what to do" surface.

---

12. Security, tiers, and access

Authentication

Supabase Auth handles signup, login, password reset, and session management. Sessions are stored as JWTs in browser cookies. Every API call to Supabase includes the JWT, which Postgres decodes to identify the user.

The `auth.users` table is managed by Supabase. A trigger (`on_auth_user_created`) auto-creates a corresponding `profiles` row when a new account is created, applying defaults.

Tiers and feature gating

Five tiers, from least to most privileged:

1. **Core** — basic features, 500-contact cap. Sphere, Pipeline, Scoreboard, Database, Events, Newsletter, Delight, Support, Settings, Resources.
2. **Managed** — 1,000-contact cap. Same features as Core.
3. **Agent** — unlimited contacts. Adds Social Scheduler (Metricool).
4. **Editor** — admin-lite for content team members.
5. **Admin** — full access including Transactions, all admin pages, team management.

The tier is stored in `auth.users.raw_app_meta_data.role`. The hook `useFeatureAccess.ts` enforces gating on:
- **Route access** — the `ROUTE_MIN_TIER` map specifies the minimum tier required to load each route.
- **Contact caps** — `CONTACT_LIMITS` map enforces the per-tier maximum.

Row-Level Security

Every table holding per-agent data has RLS enabled with policies of the form:

```
CREATE POLICY "agents see own contacts" ON contacts
  FOR ALL TO authenticated
  USING (agent_id = auth.uid())
  WITH CHECK (agent_id = auth.uid());

CREATE POLICY "admins see all contacts" ON contacts
  FOR ALL TO authenticated
  USING (get_current_user_role() = 'admin');
```

This means even if a bug in the React code tried to fetch another agent's data, Postgres would return zero rows. The security guarantee is at the database level, not the application level.

Public surfaces

A small number of routes work without auth — and they're carefully scoped:

- `/event/:slug` — public RSVP page for a specific event. Reads via the `get_public_event_agent_profile()` SECURITY DEFINER function which exposes only the fields needed for branding (name, headshot, brokerage, license states) — nothing sensitive.
- `/support/articles/:slug` — knowledge base articles. Tier-gated articles (`min_tier = 'agent'` or `'admin'`) are hidden from non-authenticated readers.

Compliance touches

- **Do-Not-Call (DNC)** — every contact has a `dnc` boolean. Monthly cron re-verifies phone numbers against the FTC registry. The UI surfaces DNC prominently (a red pill on every card, a disabled call button) — the agent can still email or text but phone calls are gated.
- **CAN-SPAM** — Newsletter footer includes the agent's office address and a privacy policy link (set in `/settings` → Profile). Required by federal law for commercial email.
- **GDPR-style data export** — `/settings` → Data & export lets the agent download their own contacts, check-ins, and newsletter history as CSV at any time.

---

13. Appendix — file map

Frontend (React)

```
src/
■■■ App.tsx                    # Route definitions
■■■ pages/                     # One file per route
■   ■■■ Index.tsx              # / (Dashboard)
■   ■■■ Pipeline.tsx           # /pipeline
■   ■■■ SphereSyncTasks.tsx    # /spheresync-tasks
■   ■■■ Scoreboard.tsx         # /scoreboard
■   ■■■ Database.tsx           # /database
■   ■■■ Settings.tsx           # /settings
■   ■■■ Newsletter.tsx         # /newsletter
■   ■■■ Events.tsx             # /events
■   ■■■ Delight.tsx            # /delight
■   ■■■ Support.tsx            # /support
■   ■■■ Transactions.tsx       # /transactions  (admin-only)
■   ■■■ ...
■■■ hooks/                     # Data fetching + business logic
■   ■■■ usePipeline.ts
■   ■■■ useSphereSyncTasks.ts
■   ■■■ usePrioritizedQueue.ts        # The 3-band queue
■   ■■■ usePrioritizedContacts.ts     # Tier helpers
■   ■■■ useCoachingState.ts           # AI Coach state
■   ■■■ useAgentIntelligence.ts       # Weekly Claude snapshot
■   ■■■ useCoaching.ts                # Check-ins + streaks
■   ■■■ useNotificationPrefs.ts       # Quiet hours, timezone
■   ■■■ useFeatureAccess.ts           # Route tier gating
■   ■■■ ... ~40 more
■■■ components/
■   ■■■ pipeline/              # Kanban board, cards, dialogs, drawer
■   ■■■ spheresync/            # ContactQuickSheet, tabs, priority queue
■   ■■■ dashboard/             # CommanderHero, StreakCard, Modules
■   ■■■ scoreboard/            # Check-in modal, KPI cards, charts
■   ■■■ settings/              # AssetUploader, section editors
■   ■■■ ...
■■■ config/
■   ■■■ pipelineStages.ts      # Pam's 7 stages — single source of truth
■   ■■■ stripe.ts              # Pricing tiers
■■■ utils/
■   ■■■ sphereSyncLogic.ts     # The letter rotation
■■■ integrations/supabase/
    ■■■ types.ts               # Auto-generated DB schema types
```

Backend (Supabase)

```
supabase/
■■■ functions/                # ~70 Deno edge functions
■   ■■■ ai-coach-agent/           # The real-time Coach
■   ■■■ generate-agent-intelligence/  # Weekly Claude synthesis
■   ■■■ compute-priority-scores/  # The temperature scorer
■   ■■■ spheresync-generate-tasks/    # Weekly task generation
■   ■■■ coaching-weekly-nudge/    # In-app check-in nudges
■   ■■■ coaching-reminder/        # Email check-in reminders
■   ■■■ delight-daily-nudge/      # Birthday/anniversary cues
■   ■■■ delight-suggest-gift/     # AI gift suggestions
■   ■■■ generate-ai-newsletter/   # Newsletter draft generation
■   ■■■ newsletter-send/          # One-off newsletter dispatch
■   ■■■ opentoclose-sync/         # External transaction sync
■   ■■■ sync-realtor-market-data/ # ZIP-level market data
■   ■■■ support-search/           # KB full-text search
■   ■■■ ...
■■■ migrations/               # ~300 schema migrations
    ■■■ 20260514000001_pam_pipeline_stages.sql   # The latest — Pam's 7 stages
```

External services

- **Vercel** — hosts the React frontend, auto-deploys on merge to `main`.
- **Supabase** — managed Postgres + Auth + Storage + Edge runtime.
- **xAI** — Grok API for high-volume AI work (intent scoring, Coach ticks).
- **Anthropic** — Claude API for high-quality synthesis (weekly intelligence, SphereSync task scoring).
- **Resend** — transactional email (newsletter, reminders, RSVPs).
- **Stripe** — subscription billing (Core / Managed / Agent tiers).
- **Metricool** — social media scheduling for the Agent tier.
- **OpenToClose** — transaction coordination platform (admin pulls closings from here).
- **Realtor.com** — ZIP-level market data feed.
- **ClickUp** — event production tasks (admin operations side).

Closing notes for partners

A few things worth knowing as you evaluate the system:

What's strongest.
- The **deterministic priority queue** (`usePrioritizedQueue`) is honest — agents can always trust the 3-band ordering because the math is rule-based, not AI. AI augments (intent scoring, Coach reasoning) but doesn't decide the order.
- The **separation of cheap-fast Grok and expensive-slow Claude** keeps the AI bill bounded while still using the right tool for each job.
- **RLS at the database level** means even bugs in the React code can't leak data across agents.
- **The cron schedule is sparse and well-named** — 17 jobs, each with a clear purpose, easy to audit.

What's transitional.
- The Coach is **running** (data is being written to `agent_coaching_state` every 2 hours during workdays) but most of its output isn't yet surfaced in the UI. `today_list`, `week_narrative`, the global `alerts[]` array, and the `AgentIntelligenceWidget` are all ready to wire — work for the next sprint.
- The pipeline rebuild to Pam's 7 stages just shipped on May 14, 2026 (PR #28). The visible behavior is correct, but a few months of operating data will tell us if `client_secured` actually gets used or if agents skip it.
- The **Coach-authored task** flow (Coach creates a SphereSync or Pipeline task) is implemented end-to-end but the UI to surface "this task came from Coach" is not yet built.

What we're not doing yet.
- No machine learning beyond what the API models provide. The system doesn't train its own models — it composes calls to Grok and Claude.
- No real-time presence / collaboration features. Each agent works alone in their own data.
- The Coach's recommendations are not benchmarked against agent outcomes yet (whether following its advice correlates with closings). That's a measurement-loop project we'd want before claiming the AI "drives results."

If you want a deeper dive on any specific subsystem — the Coach prompts, the priority formula's edge cases, the cron failure modes — those are good follow-up conversations.